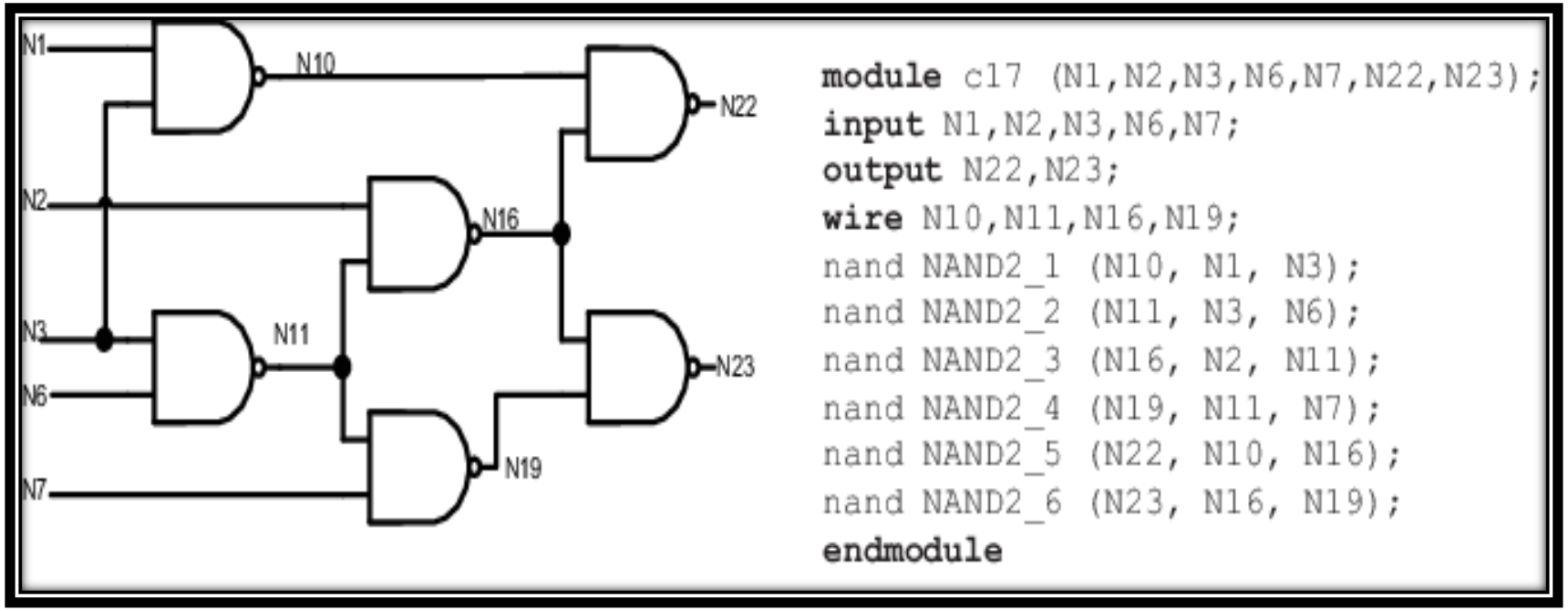
Digital Systems Design : BECE102L

Blocking and Non-Blocking Assignment in Behavioural Modelling




```
module c17 (N1,N2,N3,N6,N7,N22,N23);
input N1,N2,N3,N6,N7;
output N22,N23;
wire N10,N11,N16,N19;
nand NAND2_1 (N10, N1, N3);
nand NAND2_2 (N11, N3, N6);
nand NAND2_3 (N16, N2, N11);
nand NAND2_4 (N19, N11, N7);
nand NAND2_5 (N22, N10, N16);
nand NAND2_6 (N23, N16, N19);
endmodule
```

Dr. Vetriveeran Rajamani,
Associate Professor Grade 1,
Department of Micro and Nano Electronics,
School of Electronics Engineering (SENSE),
VIT Vellore.

The assignment statements so far

	Continuous Assignment	Procedural Assignment
Operation	=	=
Where	using stand-alone *assign* statement	inside of *always* and *initial*

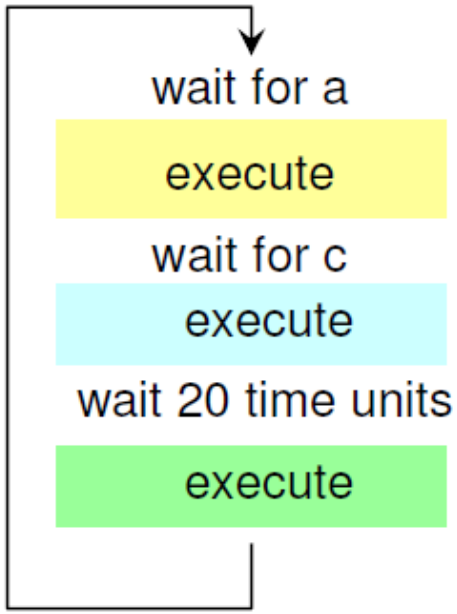
Example	`wire q;`
	`reg a, b;`
	`assign q = a & b;`
		`wire q;`
		`reg a, b;`
		`always @(b)`
		`  a = b + 5;`

Valid LHS	net (wire)	reg
Valid RHS	expression of net or reg	expression of net or reg
Evaluated	when any part of RHS changes	**procedural execution**

What is 'procedural execution' ?

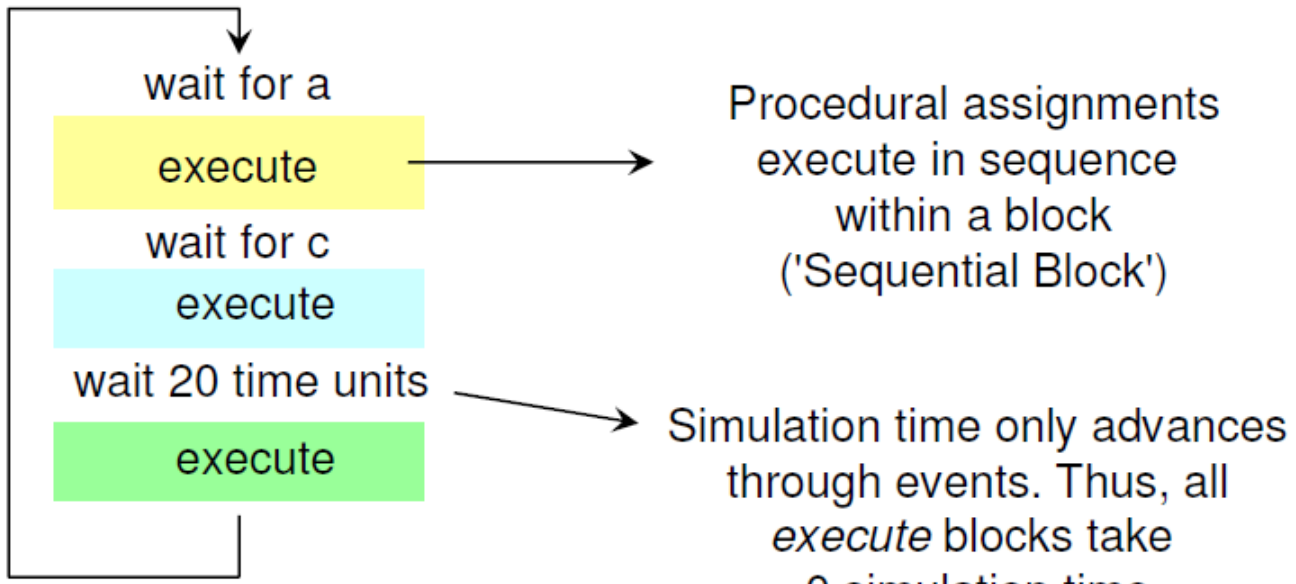
- An always block is either executing, or it is waiting for an event from its sensitivity list
- When it is executing, it proceeds in a sequential fashion through the statements of the code, until it runs into an *event control statement* (@, wait, #)

```
always begin
  @(upedge a);
  b = a + 1;
  wait (c == 3);
  #20 b = a + 1;
end
```



What is 'procedural execution' ?

- An always is either executing, or it is waiting for an event from its *sensitivity list*
- When it is executing, it proceeds in a sequential fashion through the statements of the code, until it runs into an *event control statement* (@, wait, #)



The assignment statements so far

	Continuous Assignment	Procedural Assignment
Operation	=	=
Wh		ays
Ex		a = b + 5;
Valid LHS	net (wire)	reg
Valid RHS	expression of net or reg	expression of net or reg
Evaluated	when any part of RHS changes	procedural execution

The full name of this assignment is:

Procedural *Blocking* Assignment

Procedural Blocking Assignment

- Execution in a sequential block cannot complete without the assignment being complete

```
reg [7:0] a, b;
always @(posedge clk) begin
    a = b + 2;
    b = a * 3;
end
```

Blocking assignment:

assignment on *a* MUST complete before assignment on *b* can start

Procedural Blocking Assignment

- Execution in a sequential block cannot complete without the assignment being complete

```
reg [7:0] a, b;
always @(posedge clk) begin
   #10 a = b + 2;
   b = a * 3;
end
```

1. **Wait 10;**
2. Read value of b and add 2 to it;
3. Assign the result to a;
4. Read the value of a and multiply by three;
5. Assign the result to b;

Procedural Blocking Assignment

- Execution in a sequential block cannot complete without the assignment being complete

```
reg [7:0] a, b;
always @(posedge clk) begin
    a = #10 b + 2;
    b = a * 3;
end
```

1. Read value of b and add 2 to it;
2. **Wait 10;**
3. Assign the result to a;
4. Read the value of a and multiply by three;
5. Assign the result to b;

Procedural Blocking Assignment

- Execution in a sequential block cannot complete without the assignment being complete

```
reg [7:0] a, b;
always @(posedge clk) begin
   #5 a = #10 b + 2;
   #5 b = a * 3;
end
```

1. **Wait 5;**
2. Read value of b and add 2 to it;
3. **Wait 10;**
4. Assign the result to a;
5. **Wait 5;**
6. Read the value of a and multiply by three;
7. Assign the result to b;

Procedural Blocking Assignment

- Execution in a sequential block cannot complete without the assignment being complete

Blocking Assignment means
a will always be assigned before b

```
reg [7:0] a, b;
always @(posedge clk) begin
   #5 a = #10 b + 2;
   #5 b = a * 3;
end
```

1. **Wait 5;**
2. Read value of b and add 2 to it;
3. **Wait 10;**
4. **Assign the result to a;**
5. **Wait 5;**
6. Read the value of a and multiply by three;
7. **Assign the result to b;**

Procedural Non-Blocking Assignment

	Continuous Assignment	Proc. Blocking Assignment	Proc. Non-Blocking Assignment
Operation	=	=	<=
Where	using stand-alone *assign* statement	inside of *always* and *initial*	inside of *always* and *initial*

Example	`wire q;`
	`reg a, b;`
	`assign q = a & b;`
		`wire q;`
		`reg a, b;`
		`always @(b)`
		`  a = b + 5;`
			`wire q;`
			`reg a, b;`
			`always @(b)`
			`  a <= 5;`

Valid LHS	net (wire)	reg	reg
Valid RHS	expression of net or reg	expression of net or reg	expression of net or reg
Evaluated	when any part of RHS changes	procedural execution	at the end of current time step

Effect of a non-blocking statement

Blocking

```
reg [7:0] a, b;

initial b = 0;
initial a = 4;

always @(a or b)
  begin
    a = b + 2;
    b = a * 3;
 end
```

Non-Blocking

```
reg [7:0] a, b;

initial b = 0;
initial a = 4;

always @(a or b)
  begin
    a <= b + 2;
    b <= a * 3;
 end
```

Effect of a non-blocking statement

Blocking

```
reg [7:0] a, b;

initial b = 0;
initial a = 4;

always @(a or b)
  begin
    a = b + 2;
    b = a * 3;
 end
```

a = 0 + 2 = 2
b = 2 * 3 = 6

Non-Blocking

```
reg [7:0] a, b;

initial b = 0;
initial a = 4;

always @(a or b)
  begin
    a <= b + 2;
    b <= a * 3;
 end
```

a = 0 + 2 = 2
b = 4 * 3 = 12

Effect of a non-blocking statement

Blocking

```
reg [7:0] a, b;

initial b = 0;
initial a = 0;

always @(a or b)
  begin
    a = b + 2;
    b = a * 3;
  end
```

a = 0 + 2 = 2
*b = 2 * 3 = 6*

Non-Blocking

```
reg [7:0] a, b;

initial b = 0;
initial a = 0;

always @(a or b)
  begin
    a <= b + 2;
    b <= a * 3;
  end
```

a = 0 + 2 = 2
*b = 4 * 3 = 12*

Non-Blocking Assignment means a and b are updated at the same time

Another example

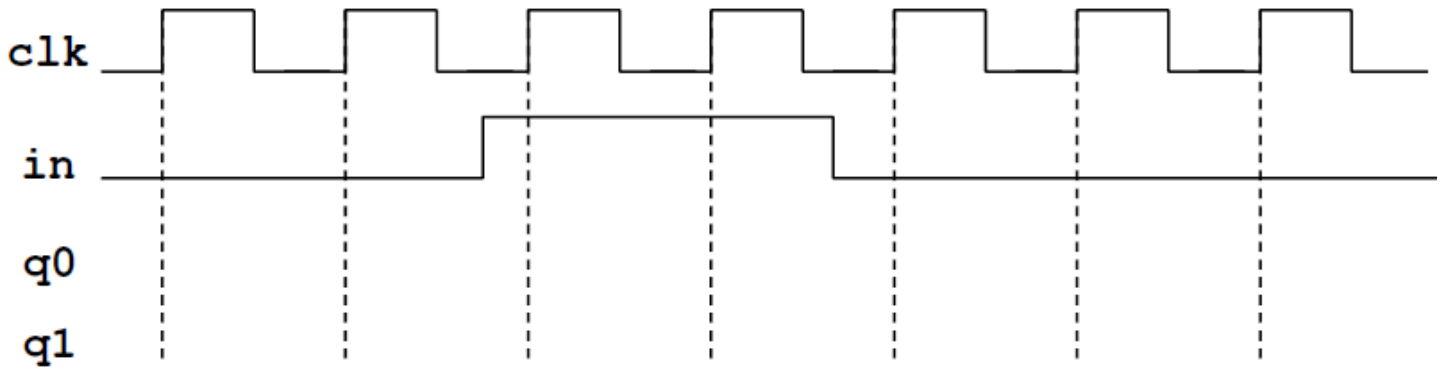
Using blocking assignments:

```
module fsm_mod(q1, q0, in, clk);
     output q1, q0;
     input clk, in;
     reg q1, q0;

     always @(posedge clk) begin
          q1 = in;
          q0 = in | q1;
     end
endmodule
```

Another example

Using blocking assignments:

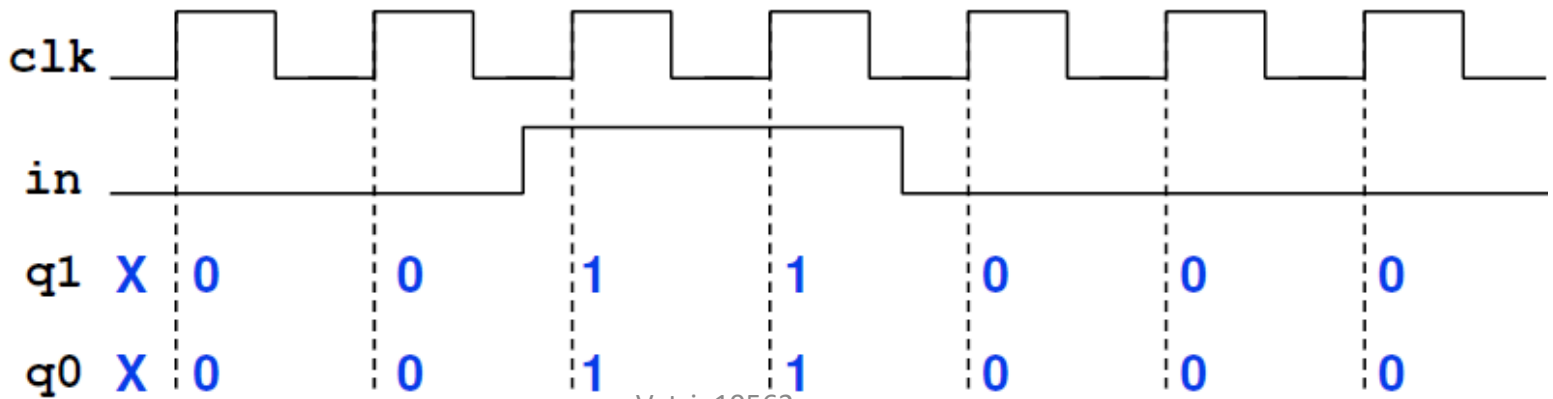
```
module fsm_mod(q1, q0, in, clk);
    output q1, q0;
    input clk, in;
    reg q1, q0;

    always @(posedge clk) begin
        q1 = in;
        q0 = in | q1;
    end
endmodule
```



Another example

Using blocking assignments:

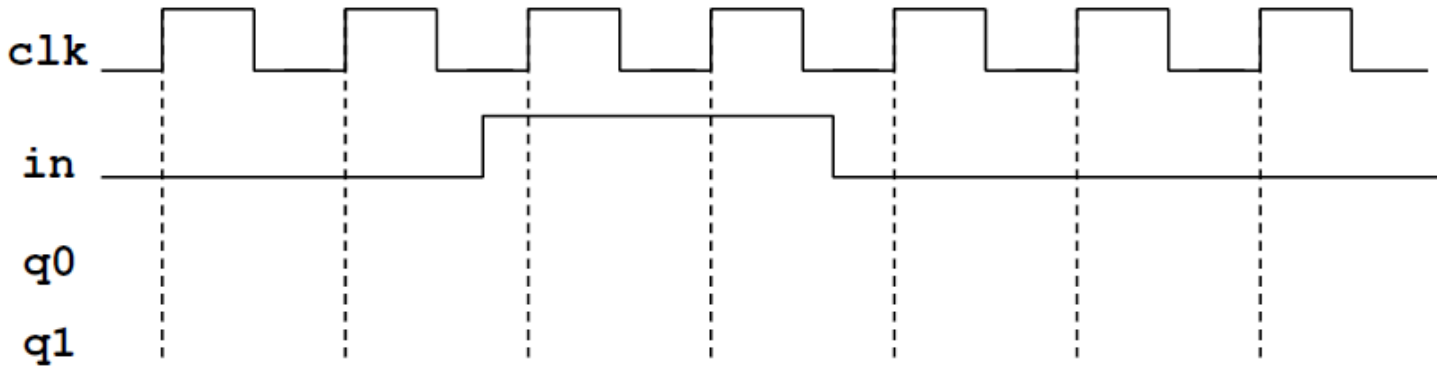
```
module fsm_mod(q1, q0, in, clk);
    output q1, q0;
    input clk, in;
    reg q1, q0;

    always @(posedge clk) begin
        q1 = in;
        q0 = in | q1;
    end
endmodule
```



Another example

Using blocking assignments:

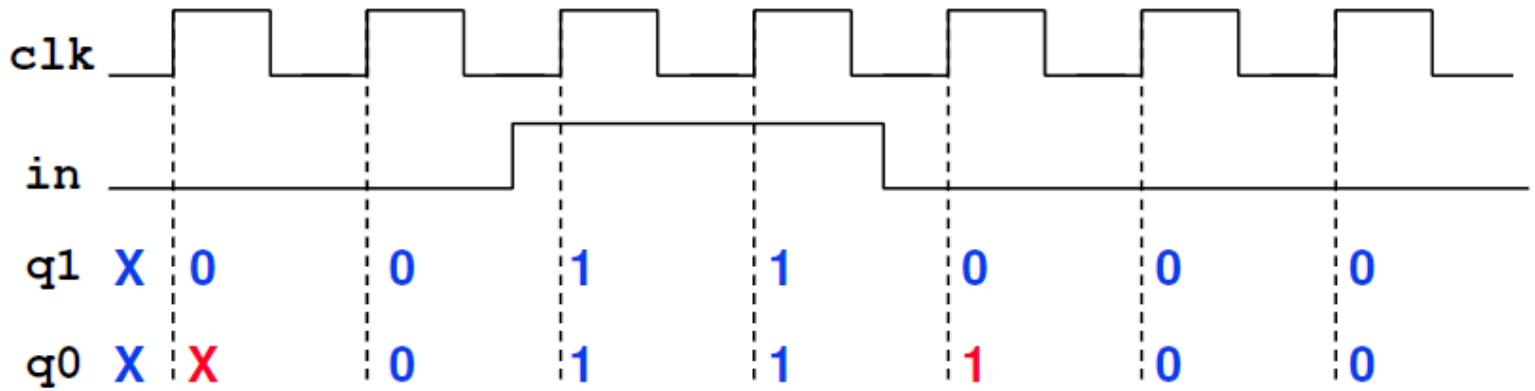
```
module fsm_mod(q1, q0, in, clk);
    output q1, q0;
    input clk, in;
    reg q1, q0;

    always @(posedge clk) begin
        q1 = in;
        q0 = in | q1;
    end
endmodule
```



Another example

Using non-blocking assignments:

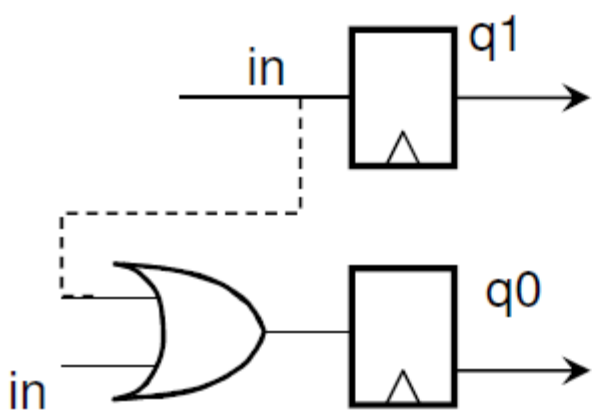
```
module fsm_mod(q1, q0, in, clk);
    output q1, q0;
    input clk, in;
    reg q1, q0;

    always @(posedge clk) begin
        q1 <= in;
        q0 <= in | q1;
    end
endmodule
```

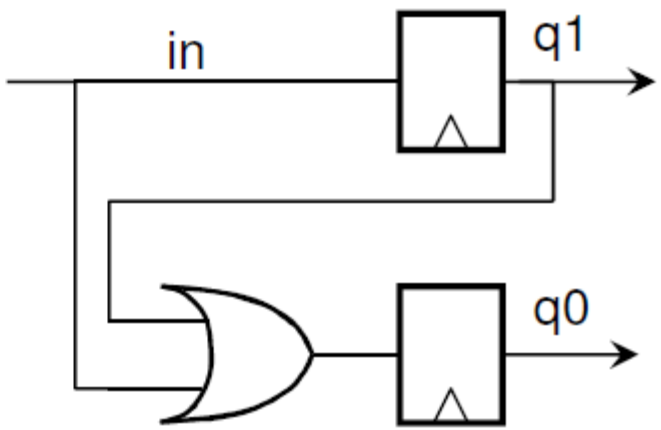


Think about the *meaning* of this behavior

```
module fsm_mod(q1, q0, in, clk);
    output q1, q0;
    input clk, in;
    reg q1, q0;

    always @(posedge clk) begin
        q1 = in;
        q0 = in | q1;
    end
endmodule
```

```
module fsm_mod(q1, q0, in, clk);
    output q1, q0;
    input clk, in;
    reg q1, q0;

    always @(posedge clk) begin
        q1 <= in;
        q0 <= in | q1;
    end
endmodule
```





I. Blocking vs. Nonblocking Assignments

- Verilog supports two types of assignments within `always` blocks, with subtly different behaviors.
- *Blocking assignment:* evaluation and assignment are immediate

```
always @ (a or b or c)
begin
  x = a | b;          1. Evaluate a | b, assign result to x
  y = a ^ b ^ c;      2. Evaluate a^b^c, assign result to y
  z = b & ~c;         3. Evaluate b&(~c), assign result to z
end
```

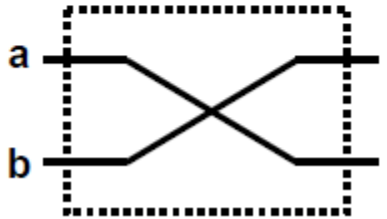
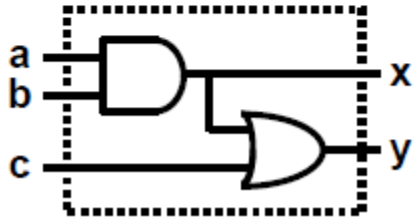
- *Nonblocking assignment:* all assignments deferred until all right-hand sides have been evaluated (end of simulation timestep)

```
always @ (a or b or c)
begin
  x <= a | b;         1. Evaluate a | b but defer assignment of x
  y <= a ^ b ^ c;     2. Evaluate a^b^c  but defer assignment of y
  z <= b & ~c;        3. Evaluate b&(~c) but defer assignment of z
end                   4. Assign x, y, and z with their new values
```

- Sometimes, as above, both produce the same result. Sometimes, not!

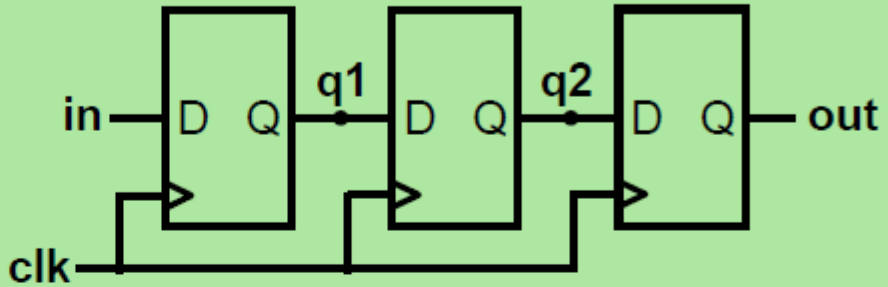
Why two ways of assigning values?

Conceptual need for two kinds of assignment (in `always` blocks):

		
Blocking: Evaluation and assignment are immediate	~~a = b~~ ~~b = a~~	x = a & b y = x | c
Non-Blocking: Assignment is postponed until all r.h.s. evaluations are done	a <= b b <= a	~~x <= a & b~~ ~~y <= x | c~~
When to use: (only in always blocks!)	Sequential Circuits	Combinational Circuits

Assignment Styles for Sequential Logic

Flip-Flop Based Digital Delay Line



- Will nonblocking and blocking assignments both produce the desired result?

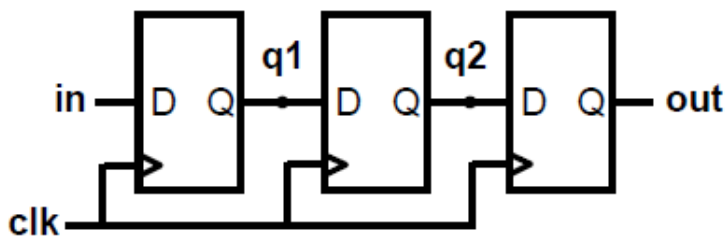
```
module nonblocking(in, clk, out);
  input in, clk;
  output out;
  reg q1, q2, out;

  always @ (posedge clk)
  begin
    q1 <= in;
    q2 <= q1;
    out <= q2;
  end

endmodule
```

```
module blocking(in, clk, out);
  input in, clk;
  output out;
  reg q1, q2, out;

  always @ (posedge clk)
  begin
    q1 = in;
    q2 = q1;
    out = q2;
  end

endmodule
```

Use Nonblocking for Sequential Logic

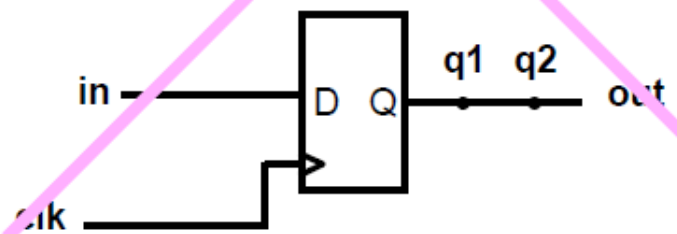
```
always @ (posedge clk)
begin
   q1 <= in;
   q2 <= q1;
   out <= q2;
end
```

"At each rising clock edge, *q1*, *q2*, and *out* simultaneously receive the old values of *in*, *q1*, and *q2*."




```
always @ (posedge clk)
begin
   q1 = in;
   q2 = q1;
   out = q2;
end
```

"At each rising clock edge, *q1* = *in*. After that, *q2* = *q1* = *in*; After that, *out* = *q2* = *q1* = *in*; Finally *out* = *in*."



- Blocking assignments do not reflect the intrinsic behavior of multi-stage sequential logic
- Guideline: use nonblocking assignments for sequential `always` blocks

Summary of Blocking and Non-Blocking Assignments

Feature	Blocking Assignment (=)	Non-Blocking Assignment (<=)
Execution	Sequential	Parallel
Use Case	Combinational logic (within always @(*) blocks)	Sequential logic (within always @(posedge clk) or negedge clk blocks)
Purpose	Modeling behavioral flow, like a software program.	Modeling hardware registers and flip-flops.
Synthesizability	Can lead to non-synthesizable code if used for sequential logic.	Ensures predictable and synthesizable sequential logic.